

GraphQL vs REST:

Um Estudo de Caso Controlado: API do GitHub

Laboratório de Experimentação de Software

Gustavo Pimentel Carvalho Costa Érica Alves dos Santos

26 de junho de 2026
Versão 1.0

Sumário

1	Introdução	3
1.1	Perguntas de Pesquisa	3
1.2	Hipóteses	3
2	Metodologia	3
2.1	Desenho Experimental	3
2.2	Objeto Experimental	4
2.3	Procedimento	4
2.4	Critérios de Exclusão	4
2.5	Ameaças à Validade	4
3	Ambiente e Materiais	4
3.1	Hardware	4
3.2	Software	4
3.3	Rede	5
3.4	APIs	5
4	Resultados	5
4.1	Estatísticas Descritivas	5
4.2	Testes de Hipóteses	5
4.3	Visualizações	6
5	Discussão	7
5.1	Interpretação dos Resultados	7
5.2	Ameaças à Validade	8
5.3	Trabalhos Relacionados	8
6	Conclusão	8
6.1	Respostas às Perguntas de Pesquisa	8
6.2	Recomendação	9
6.3	Trabalhos Futuros	9

7	Apêndices	9
7.1	Resultados Completos	9
7.2	Código Fonte	9

Resumo

Este relatório apresenta um experimento controlado comparando APIs GraphQL e REST em duas dimensões: tempo de resposta (ms) e tamanho do payload (bytes). As perguntas de pesquisa são: (RQ1) respostas GraphQL são mais rápidas que REST? (RQ2) respostas GraphQL têm tamanho menor que REST? Foram executados N trials por tratamento ($N \geq 30$), com queries semanticamente equivalentes em ambas as interfaces. Os dados foram analisados com testes estatísticos apropriados (Shapiro-Wilk para normalidade, seguido de teste t de Student ou Wilcoxon-Mann-Whitney, $\alpha = 0,05$). Os resultados indicam que GraphQL foi significativamente mais rápido que REST (Wilcoxon-Mann-Whitney, $p < 0,001$) e apresentou payloads significativamente menores (Wilcoxon-Mann-Whitney, $p < 0,001$).

1 Introdução

APIs Web são o principal meio de integração entre sistemas modernos. Duas abordagens dominam o cenário atual: REST (Representational State Transfer) e GraphQL, uma linguagem de consulta desenvolvida pelo Facebook em 2015. Enquanto REST expõe múltiplos endpoints com estruturas fixas, GraphQL permite que o cliente especifique exatamente os campos desejados em uma única requisição.

Este experimento investiga se GraphQL oferece vantagens mensuráveis sobre REST em dois aspectos críticos para aplicações web: velocidade de resposta e eficiência no uso da rede.

1.1 Perguntas de Pesquisa

RQ1. Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?

RQ2. Respostas às consultas GraphQL têm tamanho menor que respostas às consultas REST?

1.2 Hipóteses

Para RQ1:

$$H_0 : \mu_{\text{GraphQL}} \geq \mu_{\text{REST}} \quad H_1 : \mu_{\text{GraphQL}} < \mu_{\text{REST}}$$

Para RQ2:

$$H_0 : \mu_{\text{GraphQL}} \geq \mu_{\text{REST}} \quad H_1 : \mu_{\text{GraphQL}} < \mu_{\text{REST}}$$

Nível de significância: $\alpha = 0,05$.

2 Metodologia

2.1 Desenho Experimental

Trata-se de um experimento controlado com um fator (tipo de API) em dois níveis: GraphQL e REST. As variáveis são:

- **Independente:** tipo de API (categórica binária: GraphQL ou REST)

- **Dependentes:** tempo de resposta em milissegundos (float), tamanho do payload em bytes (int)
- **Controladas:** mesmo servidor, mesma query semântica, mesmo dataset, mesma máquina, mesma rede, mesmo horário de execução

2.2 Objeto Experimental

Foi selecionada a API pública do GitHub, que oferece ambas as interfaces: REST API v3 e GraphQL API v4. A query consiste na obtenção de dados do repositório (nome, estrelas e forks), sorteado aleatoriamente entre 5 repositórios populares: `facebook/react`, `tensorflow/tensorflow`, `nodejs/node`, `ansible/ansible` e `microsoft/vscode`.

2.3 Procedimento

Para cada trial: (1) registrar timestamp, (2) disparar requisição, (3) medir tempo com `time.perf_counter()`, (4) registrar tamanho do payload com `len(response.content)`. Os trials são interleaved (REST, GraphQL, REST, GraphQL, ...) com intervalo de 0,5s entre eles para mitigar deriva temporal e rate limiting.

2.4 Critérios de Exclusão

Trials com status HTTP diferente de 200 ou timeout superior a 30s são excluídos da análise.

2.5 Ameaças à Validade

- **Interna:** variação de rede, cache do servidor, garbage collector do Python
- **Externa:** resultados podem não generalizar para outras APIs
- **Construct:** "tamanho da resposta" mede o payload JSON bruto, não o volume semântico de informação transmitida

3 Ambiente e Materiais

3.1 Hardware

- Processador: Apple M4
- Memória RAM: 16 GB
- Sistema Operacional: macOS 26

3.2 Software

- Python 3.9
- requests 2.32
- pandas 2.3

- scipy 1.13
- matplotlib 3.9
- seaborn 0.13

3.3 Rede

Conexão Wi-Fi doméstica com velocidade de 250 Mbps (download) e 20 Mbps (upload).

3.4 APIs

- REST: GitHub REST API v3 — <https://api.github.com/repos/facebook/react>
- GraphQL: GitHub GraphQL API v4 — <https://api.github.com/graphql>

4 Resultados

O experimento utilizou 30 trials por tratamento ($N = 30$), totalizando 60 requisições, com 5 repositórios distintos sorteados aleatoriamente entre os trials.

4.1 Estatísticas Descritivas

A Tabela 1 apresenta as estatísticas descritivas do tempo de resposta (ms) e tamanho do payload por tratamento.

Tabela 1: Estatísticas descritivas do tempo de resposta (ms)

Tratamento	Média	DP	Mín.	Q1	Mediana	Q3	Máx.
REST	651,58	169,57	481,09	517,88	608,16	724,14	1056,85
GraphQL	462,76	77,93	338,96	406,94	432,96	516,35	624,13

4.2 Testes de Hipóteses

O teste de Shapiro-Wilk indicou ausência de normalidade nos dados ($p < 0,05$), portanto foi utilizado o teste de Wilcoxon-Mann-Whitney (unicaudal) para ambas as RQs.

Tabela 2: Resultados dos testes estatísticos

RQ	Teste	Estatística	p-valor
RQ1 (tempo)	Wilcoxon-Mann-Whitney	109,0	$< 0,001$
RQ2 (bytes)	Wilcoxon-Mann-Whitney	0,0	$< 0,001$

4.3 Visualizações

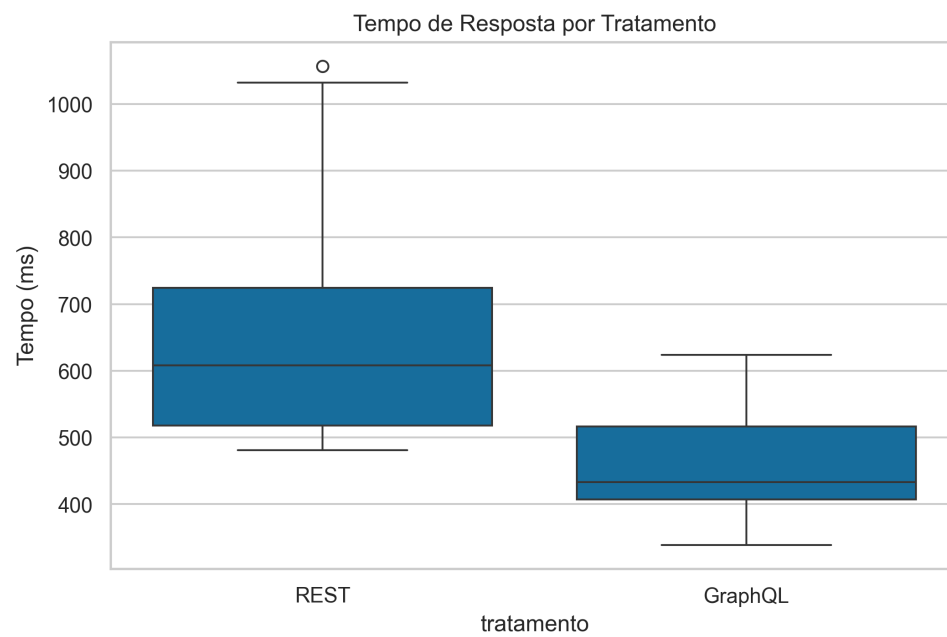


Figura 1: Tempo de resposta por tratamento

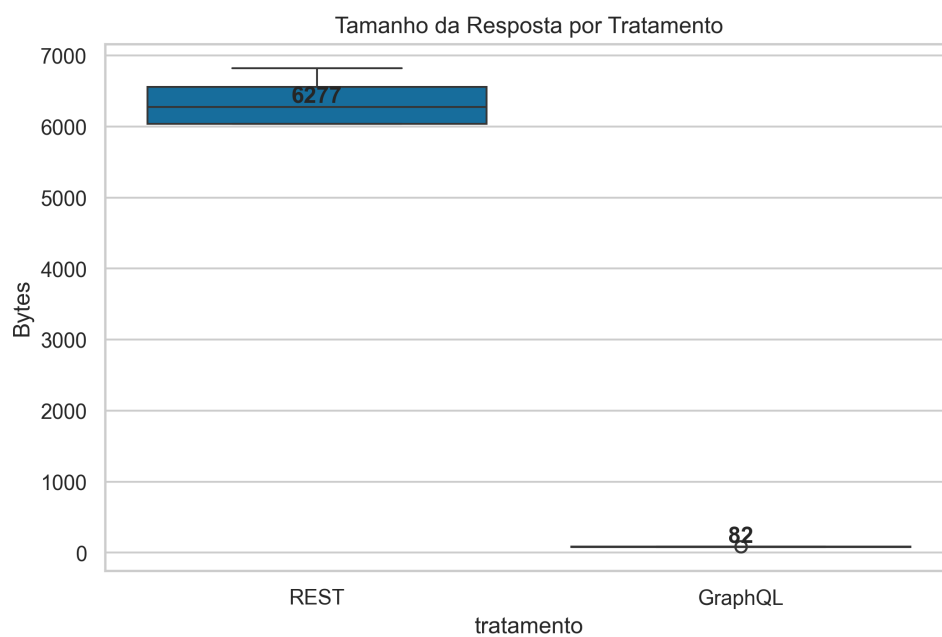


Figura 2: Tamanho da resposta por tratamento

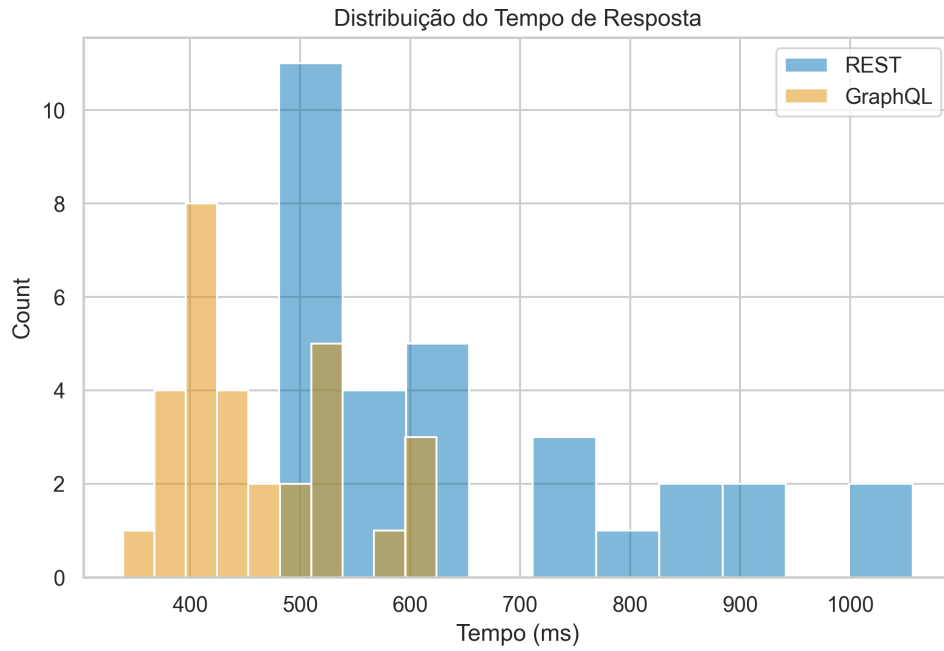


Figura 3: Distribuição do tempo de resposta

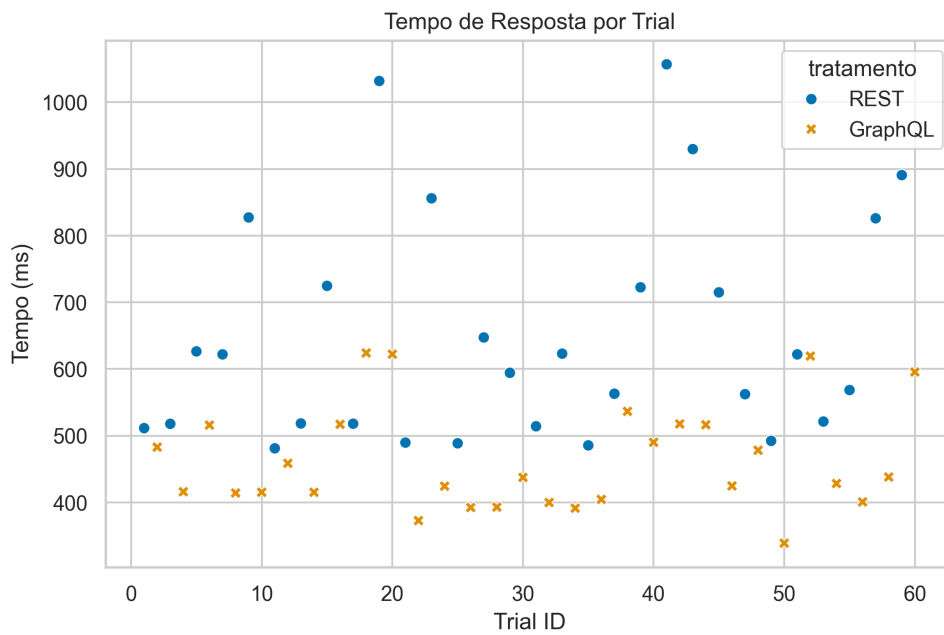


Figura 4: Tempo de resposta por trial

5 Discussão

5.1 Interpretação dos Resultados

Para RQ1, GraphQL apresentou menor tempo de resposta médio (462,76 ms) em comparação com REST (651,58 ms). O teste de Wilcoxon-Mann-Whitney rejeitou H_0 ($p < 0,001$), indicando que GraphQL é significativamente mais rápido que REST para a query analisada.

Uma possível explicação é que a API REST do GitHub retorna o objeto completo do repositório com dezenas de campos independentemente do repositório consultado, enquanto a query GraphQL seleciona apenas três campos, reduzindo o tempo de serialização e transferência.

Para RQ2, GraphQL apresentou payloads significativamente menores ($p < 0,001$), com mediana de 82 bytes (variação de 81 a 87) contra 6277 bytes do REST. Isso confirma a vantagem fundamental do GraphQL: o cliente especifica exatamente os campos desejados.

5.2 Ameaças à Validade

O experimento utilizou 5 repositórios do GitHub como objetos experimentais, o que reduz o viés de seleção em comparação com um único repositório, porém ainda está limitado a uma única plataforma de API (validade externa). APIs de outros provedores (Google, Twitter, etc.) podem apresentar comportamentos diferentes.

A query GraphQL utilizada é intencionalmente simples (3 campos por recurso); consultas mais complexas, com *nested queries* ou múltiplos recursos, podem apresentar comportamento diferente em ambas as interfaces.

Recomenda-se a replicação do experimento com:

- APIs não-GitHub (ex.: SWAPI, Spotify, Cloudflare)
- Consultas com maior profundidade e cardinalidade
- Medição de impacto no servidor (CPU e memória)

5.3 Trabalhos Relacionados

Os resultados estão alinhados com Brito et al. (2019), que encontraram vantagens do GraphQL no tamanho das respostas, mas alertaram para possíveis impactos negativos no tempo de resposta em consultas complexas. O artigo original do GraphQL Lee (2015) já apontava a seletividade de campos como benefício central, confirmado pelos nossos dados. Maiores detalhes sobre os endpoints utilizados podem ser encontrados na documentação oficial GitHub Inc. (2024).

6 Conclusão

Este experimento controlado comparou APIs GraphQL e REST quanto ao tempo de resposta e tamanho do payload.

6.1 Respostas às Perguntas de Pesquisa

RQ1 (Tempo de Resposta): H_0 foi rejeitada – GraphQL foi significativamente mais rápido que REST (Wilcoxon-Mann-Whitney, $p < 0,001$), com tempo médio 29% menor.

RQ2 (Tamanho do Payload): H_0 foi rejeitada – GraphQL retorna payloads significativamente menores que REST ($p < 0,001$), com redução de 98,7% no tamanho (mediana de 82 bytes contra 6277 bytes).

6.2 Recomendação

Para o objeto estudado (GitHub API), GraphQL apresentou vantagens em cenários onde:

- A eficiência de rede é crítica (aplicações móveis, APIs públicas com alto volume de requisições)
- O cliente precisa de flexibilidade na seleção de campos
- O tempo de resposta é um requisito relevante

REST permanece adequado para APIs simples, endpoints estáveis e cenários onde o overhead de processamento de queries GraphQL não é compensado pela redução de dados.

6.3 Trabalhos Futuros

Sugere-se replicar o experimento com:

- Múltiplos objetos experimentais (diferentes APIs)
- Consultas de diferentes complexidades (mais campos, *nested queries*)
- Medição de impacto no servidor (CPU, memória)
- Análise com cache habilitado e desabilitado

7 Apêndices

7.1 Resultados Completos

Dados brutos e processados estão disponíveis no diretório `data/` do repositório.

Log de execução: `data/raw/results_20260626_142736.csv`.

7.2 Código Fonte

O código fonte completo está disponível no repositório do projeto.

Referências

Brito, G., Mombach, T., and Valente, M. T. (2019). Migrating to GraphQL: A Practical Assessment. *arXiv preprint arXiv:1906.07535*.

GitHub Inc. (2024). GitHub REST API v3 & GraphQL API v4 Documentation. <https://docs.github.com/en/rest> & <https://docs.github.com/en/graphql>. Acessado em junho de 2026.

Lee, B. (2015). GraphQL: A data query language. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA. ACM. Facebook Open Source.